

WEEKLY ASSESSMENT - 5

1. Write a function that accepts variable-length arguments (*args).

The *args parameter allows a function to accept any number of positional arguments, which are packed into a tuple inside the function:

```
def sum_all_numbers(*args):
    total = 0
    for num in args:
        total += num
    return total

print(sum_all_numbers(10, 20, 30)) # Output: 60
print(sum_all_numbers(5, 5))      # Output: 10
```

2. Create a function that accepts keyword arguments (kwargs) and prints them.**

The **kwargs parameter collects any arbitrary named keyword arguments passed into the function as a standard dictionary:

```
def print_user_profile(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_user_profile(name="Ajay", course="AI", status="Student")
# Output:
# name: Ajay
# course: AI
# status: Student
```

3. Write a recursive function to calculate the factorial of a number.

A recursive function solves a problem by calling itself, using a defined base case to terminate execution safely:

```
def calculate_factorial(n):
    # Base case
    if n == 0 or n == 1:
        return 1
    # Recursive case
    else:
        return n * calculate_factorial(n - 1)

print(calculate_factorial(5)) # Output: 120
```

4. Explain the difference between Global and Local scope with a code example.

Scope Type	Definition	Accessibility
Global Scope	Variables declared outside of any function scope blocks.	Accessible from anywhere inside the program script.
Local Scope	Variables declared strictly inside a function's code body.	Accessible only inside that specific function block.

```
global_var = "I am global"

def scope_demo_function():
    local_var = "I am local"
    print(global_var) # Accessible
    print(local_var)  # Accessible

scope_demo_function()
# print(local_var)      # Raises NameError outside the function
```

5. Write a lambda function to sort a list of tuples based on the second element.

Anonymous lambda equations can serve as key-extractors for sequencing filters like `sorted()`:

```
data_tuples = [('apple', 3), ('banana', 1), ('cherry', 2)]

# Sorting using the second element of each tuple (index 1)
sorted_data = sorted(data_tuples, key=lambda x: x[1])

print(sorted_data) # Output: [('banana', 1), ('cherry', 2),
('apple', 3)]
```

6. Create a "Filter" function to extract prime numbers from a list.

This implementation couples a Boolean prime validation check alongside Python's built-in `filter()` structure:

```
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True

numbers_list = [2, 4, 5, 9, 11, 15, 17, 20]
```

```
prime_numbers = list(filter(is_prime, numbers_list))
```

```
print(prime_numbers) # Output: [2, 5, 11, 17]
```

7. What is a "Docstring" and how do you access it at runtime?

A Docstring (documentation string) is a string literal placed as the first statement in a function, class, or module to document its purpose. It can be dynamically evaluated at runtime via the `__doc__` attribute or using the `help()` utility:

```
def my_sample_function():  
    """This function calculates mathematical constants."""  
    pass
```

```
# Accessing at runtime  
print(my_sample_function.__doc__)  
# Output: This function calculates mathematical constants.
```

8. Write a function that returns multiple values.

Python allows functions to pack multiple return elements separated by commas, creating a tuple data structure behind the scenes:

```
def get_min_max_values(numbers):  
    low = min(numbers)  
    high = max(numbers)  
    return low, high # Returns values packed as a tuple  
  
min_val, max_val = get_min_max_values([4, 1, 9, 3, 12])  
print(f"Min: {min_val}, Max: {max_val}") # Output: Min: 1, Max: 12
```

9. Demonstrate the use of `nonlocal` in a nested function.

The `nonlocal` keyword explicitly instructs Python to modify a variable declared inside an enclosing outer scope rather than initializing a localized instance:

```
def outer_scope_function():  
    counter = 0  
  
    def inner_scope_function():  
        nonlocal counter  
        counter += 1  
        return counter  
  
    return inner_scope_function  
  
increment = outer_scope_function()
```

```
print(increment()) # Output: 1
print(increment()) # Output: 2
```

10. Create a function decorator that prints "Starting..." before any function runs.

A function decorator acts as a wrapper block targeting functional execution scopes without direct internal source alterations:

```
def startup_decorator(original_func):
    def wrapper_func(*args, **kwargs):
        print("Starting...")
        return original_func(*args, **kwargs)
    return wrapper_func

@startup_decorator
def run_core_process():
    print("The core script is processing.")

run_core_process()
# Output:
# Starting...
# The core script is processing.
```